

Logic, Programs, and **Curry-Howard Correspondence**

Propositions as Types, Proofs as Programs

Ruize Ma

Part I. Logic

Propositions:

$$A, B, C ::= p \mid A \rightarrow B \mid A \wedge B \mid A \vee B$$

Sequent judgment:

$$\Gamma \vdash A$$

Γ is a context of assumptions; A is the goal formula.

Structural Reading of Sequents

Think of a derivation as a tree of rule applications.

If a rule has premises and a conclusion, it is written:

$$\frac{(\text{premise 1} \quad \text{premise 2})}{(\text{conclusion})}$$

We will instantiate this with implication/and/or/forall rules.

Assumption Rules

$$\frac{}{(\Gamma; A \vdash A)} \quad (\text{Assup})$$

A trivial rule: if we assume A, then we can conclude A.

Implication Rules

$$\frac{(\Gamma; A \vdash B)}{(\Gamma \vdash A \rightarrow B)} \quad (\rightarrow I)$$

$$\frac{(\Gamma \vdash A \rightarrow B \quad \Gamma \vdash A)}{(\Gamma \vdash B)} \quad (\rightarrow E)$$

Conjunction Rules

$$\frac{(\Gamma \vdash A \quad \Gamma \vdash B)}{(\Gamma \vdash A \wedge B)} \quad (\wedge I)$$

$$\frac{(\Gamma \vdash A \wedge B)}{(\Gamma \vdash A)} \quad (\wedge E_1)$$

$$\frac{(\Gamma \vdash A \wedge B)}{(\Gamma \vdash B)} \quad (\wedge E_2)$$

Disjunction Rules

$$\frac{(\Gamma \vdash A)}{(\Gamma \vdash A \vee B)} \quad (\vee I_1)$$

$$\frac{(\Gamma \vdash B)}{(\Gamma \vdash A \vee B)} \quad (\vee I_2)$$

$$\frac{(\Gamma \vdash A \vee B) \quad \Gamma; A \vdash C \quad \Gamma; B \vdash C}{(\Gamma \vdash C)} \quad (\vee E)$$

Law of Excluded Middle (LEM)

$$\overline{(\vdash A \vee \neg A)} \quad (\text{LEM})$$

Note, Negation is syntactic sugar:

$$\neg P := P \rightarrow \perp$$

Derivation Example 1

Target formula:

$$\vdash A \rightarrow (B \rightarrow A)$$

Sketch:

$$\frac{\frac{(A, B \vdash A)}{(A \vdash B \rightarrow A)}}{\vdash A \rightarrow (B \rightarrow A)}$$

Derivation Example 2

$$P \rightarrow Q, Q \rightarrow R \vdash P \rightarrow R$$

$$\frac{\frac{(P \rightarrow Q, Q \rightarrow R, P \vdash Q \rightarrow R)}{(P \rightarrow Q, Q \rightarrow R, P \vdash Q)} \quad \frac{(P \rightarrow Q, Q \rightarrow R, P \vdash P \rightarrow Q) (P \rightarrow Q, Q \rightarrow R, P \vdash P)}{(P \rightarrow Q, Q \rightarrow R, P \vdash Q)}}{(P \rightarrow Q, Q \rightarrow R, P \vdash R)} \quad \frac{}{(P \rightarrow Q, Q \rightarrow R \vdash P \rightarrow R)}$$

First-Order Logic: forall

Quantified formulas:

$$\forall x.P(x)$$

Rules:

$$\frac{(\Gamma \vdash P(x))}{(\Gamma \vdash \forall x.P(x))} \text{ where } x \text{ is not free in } \Gamma \quad (\forall I)$$
$$\frac{(\Gamma \vdash \forall x.P(x))}{(\Gamma \vdash P(t))} \quad (\forall E)$$

Due to time limitations, we will not discuss existential quantifiers.

Part II. Programs

Why use functional languages here?

- Functions are first-class, so implication-like behavior is explicit.
- Types are precise, so logical structure is visible in programs.
- Lambda calculus gives a formal core close to proof rules.

Our path in this part:

1. start with higher-order functions (build intuition);
2. move to product/sum/function types;
3. then introduce lambda calculus and STLC (formal view).

Higher-Order Functions

```
applyTwice :: (a -> a) -> a -> a
applyTwice f x = f (f x)
```

```
compose :: (b -> c) -> (a -> b) -> a -> c
compose f g x = f (g x)
```

```
def apply_twice(f, x):
    return f(f(x))
```

```
auto apply_twice = [](auto f, auto x) { return f(f(x)); };
```

Functions are inputs/outputs of other functions.

Currying

```
add :: Int -> Int -> Int
```

```
add x y = x + y
```

```
def add(x):
```

```
    return lambda y: x + y
```

```
auto add = [](int x) { return [x](int y) { return x + y; }; };
```

UnCurried: $(\text{Int}, \text{Int}) \rightarrow \text{Int}$.

Curried: $\text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$ ($\rightarrow$ is right-associative).

Product Types

```
proj1 :: (a, b) -> a
```

```
proj1 (x, _) = x
```

```
proj2 :: (a, b) -> b
```

```
proj2 (_, y) = y
```

```
p = (42, "ok")    # tuple as product
```

```
left = p[0]
```

```
std::pair<int, std::string> p{42, "ok"};
```

```
auto left = p.first;
```

Constructor stores both components simultaneously.

Sum Types

```
data Either a b = Left a | Right b
```

```
std::variant<int, std::string> v = 42;
```

Interpretation: value is from the left branch or the right branch.

Untyped Lambda Calculus

Syntax:

$$t ::= x \mid (\lambda x. t) \mid tt$$

Alpha-renaming: bound variable names do not matter.

$$\lambda x. x \equiv \lambda y. y$$

Beta-reduction: function application substitutes the argument.

$$(\lambda x. x)y \rightarrow y \quad (\lambda x. \lambda y. x)ab \rightarrow a$$

Eta-conversion: a function is determined by its behavior.

$$\lambda x. f(x) \equiv f$$

Simply Typed Lambda Calculus (STLC)

Types:

$$T ::= \alpha \mid T \rightarrow T \mid T \times T \mid T + T$$

Typing judgment:

$$\Gamma \vdash t : T$$

Now terms are constrained by types.

STLC Typing Rules

$$\frac{((x : A) \in \Gamma)}{(\Gamma \vdash x : A)} \quad (\text{var})$$

$$\frac{(\Gamma; x : A \vdash t : B)}{(\Gamma \vdash \lambda x.t : A \rightarrow B)} \quad (\text{abs})$$

$$\frac{(\Gamma \vdash f : A \rightarrow B \quad \Gamma \vdash a : A)}{(\Gamma \vdash f(a) : B)} \quad (\text{app})$$

Program Derivation Example

Term: $\lambda x.\lambda y.x$

Type: $A \rightarrow (B \rightarrow A)$

Derivation shape:

$$\frac{\frac{x : A, y : B \vdash x : A}{x : A \vdash \lambda y.x : B \rightarrow A}}{\vdash \lambda x.\lambda y.x : A \rightarrow (B \rightarrow A)}$$

Have you noticed that this corresponds to some earlier formula?

Part III. Curry-Howard Correspondence

Do you find any similarities between the logic rules and the typing rules?

Rule Correspondence Table I ($\rightarrow$, and)

Logic (sequent style)	Lambda (sequent style)
(Assup) $\frac{}{(\Gamma, A \vdash A)}$	(Assup) $\frac{}{(\Gamma, x : A \vdash x : A)}$
($\rightarrow$ Intro) $\frac{(\Gamma, A \vdash B)}{(\Gamma \vdash A \rightarrow B)}$	(abs) $\frac{(\Gamma, x : A \vdash t : B)}{(\Gamma \vdash \lambda x.t : A \rightarrow B)}$
($\rightarrow$ Elim) $\frac{(\Gamma \vdash A \rightarrow B \quad \Gamma \vdash A)}{(\Gamma \vdash B)}$	(app) $\frac{(\Gamma \vdash f : A \rightarrow B \quad \Gamma \vdash a : A)}{(\Gamma \vdash f(a) : B)}$
(and Intro) $\frac{(\Gamma \vdash A \quad \Gamma \vdash B)}{(\Gamma \vdash A \wedge B)}$	(pair Intro) $\frac{(\Gamma \vdash u : A \quad \Gamma \vdash v : B)}{(\Gamma \vdash (u, v) : A \times B)}$
(and Elim 1) $\frac{(\Gamma \vdash A \wedge B)}{(\Gamma \vdash A)}$	(proj 1) $\frac{(\Gamma \vdash p : A \times B)}{(\Gamma \vdash \text{fst}(p) : A)}$

Rule Correspondence Table II (and, or)

Logic (sequent style)	Lambda (sequent style)
(and Elim 2) $\frac{(\Gamma \vdash A \wedge B)}{(\Gamma \vdash B)}$	(proj 2) $\frac{(\Gamma \vdash p : A \times B)}{(\Gamma \vdash \text{snd}(p) : B)}$
(or Intro 1) $\frac{(\Gamma \vdash A)}{(\Gamma \vdash A \vee B)}$	(+ Intro 1) $\frac{(\Gamma \vdash a : A)}{(\Gamma \vdash \text{left}(a) : A + B)}$
(or Intro 2) $\frac{(\Gamma \vdash B)}{(\Gamma \vdash A \vee B)}$	(+ Intro 2) $\frac{(\Gamma \vdash b : B)}{(\Gamma \vdash \text{right}(b) : A + B)}$
(or Elim) $\frac{(\Gamma \vdash A \vee B \quad \Gamma \vdash A \rightarrow C \quad \Gamma \vdash B \rightarrow C)}{(\Gamma \vdash C)}$	(+ Elim) $\frac{(\Gamma \vdash e : A + B \quad \Gamma \vdash f : A \rightarrow C \quad \Gamma \vdash g : B \rightarrow C)}{(\Gamma \vdash \delta(e, f, g) : C)}$

Oops! Do you notice that we have missed something?

Curry-Howard Correspondence !

Judgment-level correspondence:

- Left (logic): $\Gamma \vdash A$
- Right (typed lambda): $\Gamma \vdash t : A$

Same context Γ , same goal formula/type A , but on the right we also show the witness term t . (Type - Proposition, Term - Proof)

Here we work in the constructive setting, so LEM is not assumed as a general rule. For any proposition A , LEM would require a term of type $A + (A \rightarrow \perp)$, which is not uniformly constructible.

At this point, we have built a correspondence between propositional logic and STLC.

What the Typing Rules Add

Compare the two judgments again:

- Logic: $\Gamma \vdash A$
- Typing: $\Gamma \vdash t : A$

Both sides contain:

- context Γ ;
- proposition/type A .

The typing side also contains a term t .

Under Curry-Howard, this term is the proof object.

Proof as Lambda Terms

Recall the earlier theorem:

$$P \rightarrow Q, Q \rightarrow R \vdash P \rightarrow R$$

In the typed view, assumptions become variables:

$$\Gamma = f : P \rightarrow Q, g : Q \rightarrow R$$

$$\frac{\frac{(\Gamma, x : P \vdash f : P \rightarrow Q) \quad (\Gamma, x : P \vdash x : P)}{(\Gamma, x : P \vdash f(x) : Q)} \quad (\Gamma, x : P \vdash g : Q \rightarrow R)}{(\Gamma, x : P \vdash g(f(x)) : R)} \quad \frac{}{(\Gamma \vdash \lambda x. g(f(x)) : P \rightarrow R)}$$

Proof Term as Compressed Derivation

The whole derivation can be represented by its root term:

$$f : P \rightarrow Q, g : Q \rightarrow R \vdash \lambda x. g(f(x)) : P \rightarrow R$$

Program reading:

given $f : P \rightarrow Q$ and $g : Q \rightarrow R$, build a function $P \rightarrow R$.

Logic reading:

given proofs of $P \rightarrow Q$ and $Q \rightarrow R$, construct a proof of $P \rightarrow R$.

The type checker can reconstruct the proof tree from the term.

Examples: Lean4 + Haskell Proof Terms

Now we instantiate each pair above with concrete terms:

- Haskell as typed functional witnesses.
- Lean4 as machine-checked proof terms.
- Some examples also provide C++/Python code for reference. However, note that their type systems are not strong enough, so the proofs are for reference only and have no practical significance.

Example 1 (Assup): $A \rightarrow A$

Haskell witness:

```
idP :: a -> a
idP x = x
```

Lean4 witness:

```
theorem ax1 : A -> A := fun a => a
```

Other:

```
def id_p(x):
    return x

auto id_p = [](auto x) { return x; };
```

Example 2 (and Elim 1): $(A \wedge B) \rightarrow A$

Haskell:

```
ax2 :: (a, b) -> a
ax2 (x, _) = x
```

Lean4:

```
theorem ax2 (A B : Prop) : A ∧ B -> A := fun h => h.left
```

C++ reference:

```
auto ax2 = [](const auto& p) { return p.first; };
```

Example 3 (and Elim 2): $(A \wedge B) \rightarrow B$

Haskell:

```
ax3 :: (a, b) -> b
ax3 (_, y) = y
```

Lean4:

```
theorem ax3 (A B : Prop) : A ∧ B -> B := fun h => h.right
```

Example 4 (or Intro 1): $A \rightarrow A \vee B$

Haskell:

```
ax4 :: a -> Either a b
ax4 x = Left x
```

Lean4:

```
theorem ax4 (A B : Prop) : A -> A v B := fun a => Or.inl a
```

C++ reference:

```
template <class A, class B> std::variant<A, B> ax4(A a) {
    return std::variant<A, B>{std::in_place_index<0>, a};
}
```


Example 5 (or Intro 2): $B \rightarrow A \vee B$

Haskell:

```
ax5 :: b -> Either a b  
ax5 y = Right y
```

Lean4:

```
theorem ax5 (A B : Prop) : B -> A v B := fun b => Or.inr b
```

Example 6 (or Elim)

Formula:

$$(A \rightarrow C) \rightarrow (B \rightarrow C) \rightarrow (A \vee B) \rightarrow C$$

Haskell:

```
ax6 :: (a -> c) -> (b -> c) -> Either a b -> c
ax6 f g e = case e of
  Left x  -> f x
  Right y -> g y
```

Example 6 (or Elim)

Lean4:

```
theorem ax6 (A B C : Prop) :  
  (A -> C) -> (B -> C) -> (A v B) -> C :=  
  (λ fl fr ab =>  
    match ab with  
    | .inl a => fl a  
    | .inr b => fr b)
```

Example 6 (or Elim)

C++ reference:

```
template <class A, class B, class C, class FL, class FR>
C ax6(FL fl, FR fr, const std::variant<A, B>& e) {
    if (std::holds_alternative<A>(e)) return fl(std::get<A>(e));
    return fr(std::get<B>(e));
}
```

Case split in proof = pattern match in program.

Example: Currying

$P \times Q \rightarrow R$ (uncurried) and $P \rightarrow Q \rightarrow R$ (curried) are equivalent.

```
toCurried :: ((p, q) -> r) -> p -> q -> r  
toCurried h p q = h (p, q)
```

```
toUncurried :: (p -> q -> r) -> (p, q) -> r  
toUncurried f (p, q) = f p q
```

```
example : (p → (q → r)) ↔ (p ∧ q → r) :=  
  Iff.intro  
    (λ f => λ h : p ∧ q => f h.1 h.2)  
    (λ f => λ (hp : p) (hq : q) => f ⟨hp, hq⟩)
```

forall and Pi-Type

Logic:

$$\forall x : A, P(x)$$

Type-theoretic form:

$$(x : A) \rightarrow P(x)$$

When codomain depends on x , this is a Pi type.

Rule Correspondence: forall / Pi

Logic (first-order)	Dependent type theory
(forall Intro) $\frac{(\Gamma \vdash P(x))}{(\Gamma \vdash \forall x : A. P(x))}$ where x is not free in Γ	(Pi Intro) $\frac{(\Gamma, x : A \vdash t : P(x))}{(\Gamma \vdash \lambda x. t : (x : A) \rightarrow P(x))}$
(forall Elim) $\frac{(\Gamma \vdash \forall x : A. P(x)) \quad \Gamma \vdash a : A}{(\Gamma \vdash P(a))}$	(Pi Elim / app) $\frac{(\Gamma \vdash f : (x : A) \rightarrow P(x)) \quad \Gamma \vdash a : A}{(\Gamma \vdash f(a) : P(a))}$

forall $x : A$, $P(x)$ corresponds to a function that returns one proof for each input x .

With Pi-Type (dependent arrow), $\lambda_{\rightarrow}$ (STLC) naturally extends to λP

Lean4 Example of forall/Pi

eg1

```
theorem forall_id (P : A -> Prop) :  
  (∀ x : A, P x -> P x) :=  
  λ x (px : P x) => px
```

eg2

```
example (α : Type u) (p q : α → Prop) :  
  (∀ x, p x → q x) → (∀ x, p x) → (∀ x, q x) :=  
  λ h hp => (λ x : α => h x (hp x))
```

One term works for every $x : A$.

Lean4 Example: Dependent Predicate

Now the result proposition changes with the input x .

```
def Even (n : Nat) : Prop :=  $\exists k : \text{Nat}, n = k + k$ 
```

```
theorem even_to_pair :
```

```
   $\forall x : \text{Nat}, \text{Even } x \rightarrow \text{Even } x \wedge \text{Even } (x + x) :=$ 
```

```
     $\lambda x \text{ hx} \Rightarrow \text{And.intro hx}$ 
```

```
      (Exists.intro x (Eq.refl (x + x)))
```

The proof is a dependent function:

```
 $\lambda x \text{ hx} \Rightarrow \text{And.intro hx (Exists.intro x (Eq.refl (x + x)))}$ 
```

`Exists.intro x ...` chooses x as the witness for `Even (x + x)`.

Lean4 Example: Uniform Construction

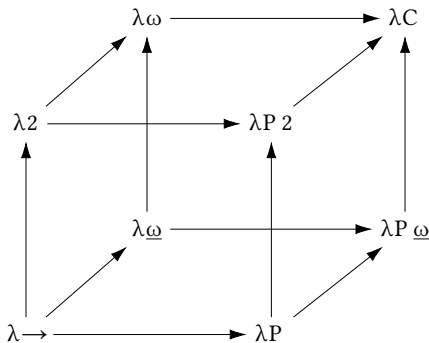
Here the returned proof is built uniformly for every x .

```
theorem double_even :  
   $\forall x : \text{Nat}, \text{Even } (x + x) :=$   
     $\lambda x \Rightarrow \text{Exists.intro } x (\text{Eq.refl } (x + x))$ 
```

`double_even 3` is a proof of `Even (3 + 3)`.

Beyond STLC: Lambda Cube

System	Logic under Curry-Howard
$\lambda_{\rightarrow}$ / STLC	intuitionistic propositional logic
λP	first-order intuitionistic predicate logic
$\lambda 2$ / System F	second-order intuitionistic propositional logic
$\lambda P 2$	part of second-order predicate logic
λC	stronger higher-order dependent type theory / Calculus of Constructions



The systems in this talk live near the lower-left part: $\lambda_{\rightarrow}$ and λP .

Thanks for Listening !